SAGE: Safe Architecture Graph Editing for Agentic Vulnerability Discovery via Constrained POMDPs

Anonymous Author(s)

ABSTRACT

Agentic security systems—compositions of language models with tools such as static analyzers, fuzzers, exploit validators, and patch generators—are increasingly deployed for automated vulnerability discovery. In practice, performance depends critically on architecture: which components exist, how they connect, and what model variants each uses. The optimal architecture is target-dependent, only partially observable during an engagement, and subject to change as both the target and security landscape evolve. Two constraints are unavoidable: budgets on computational resources, and safety requirements that prevent prompt-injection-driven tool misuse.

We introduce Sage, a framework for online architecture adaptation of agentic security systems. We formalize adaptation as a Constrained Partially Observable Markov Decision Process where the state includes the agent-tool graph, trace-derived features, and budget-risk meters, while actions correspond to reversible graph edits: adding or removing nodes, rewiring edges, adjusting parameters, and switching model variants. Safety is enforced through two layers: hard structural constraints via projection into an allowed-architecture set mandating guards between untrusted inputs and sensitive tools, and soft risk constraints via Constrained MDP optimization with Lagrangian updates.

We contribute: (1) a formal C-POMDP specification of architecture adaptation with typed graph actions; (2) a two-layer safety architecture separating structural invariants from risk budgets; (3) learning algorithms from constrained contextual bandits to full constrained RL; and (4) evaluation on synthetic benchmarks (9.6× improvement over static baselines) and a calibrated simulation study on CyberSecEval task distributions demonstrating that Sage's mechanisms achieve higher safety than static and heuristic baselines. We provide a reproducible open-source implementation, define an evaluation protocol for future real-system testing, and discuss limitations and responsible deployment.

CCS CONCEPTS

• **Security and privacy** → **Software security engineering**; • **Computing methodologies** → **Reinforcement learning**; *Planning under uncertainty*;

KEYWORDS

Agentic security, architecture adaptation, constrained reinforcement learning, contextual bandits, prompt injection, tool misuse prevention, vulnerability discovery

Conference'17, July 2017, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Security automation has entered a fundamentally new era. What once consisted of a scanner producing a static report has evolved into *agentic workflows*: directed graphs of specialized components—large language models, static analyzers, fuzzers, exploit validators, patch generators, evidence stores, and policy guards—that interact through tool calls and structured memory. Frameworks for tool-using and multi-agent composition such as ReAct [33], Toolformer [25], and AutoGen [30] have accelerated this trend in general-purpose AI applications. Security-specific systems such as PentestGPT [6] demonstrate that decomposing penetration testing into interacting modules can help maintain context and improve task completion relative to monolithic prompting strategies.

A central observation motivates this work: *architecture is destiny*. Two systems using the same underlying language model can produce radically different outcomes depending on how components are routed, how memory is scoped, where guards are placed, which model or tool variant is selected for each subtask, and when the system decides to probe versus patch. Consider a concrete scenario: an agentic system performing vulnerability discovery against a web application. Early in the engagement, broad reconnaissance with lightweight models maximizes coverage per dollar spent. As promising attack vectors emerge, the system benefits from switching to more capable models for precise validation. If the target begins exhibiting signs of prompt injection attempts in its responses, inserting additional guards becomes critical. A static architecture cannot respond to these dynamics; it must be designed conservatively enough to handle the worst case, sacrificing efficiency in typical cases.

This architecture adaptation problem exhibits three challenging properties. First, targets are only *partially observable*: one never truly sees the complete state of the system under test, only the outputs of probes and tools. Second, the environment is *nonstationary*: defenses change, software builds roll forward, rate limits activate, and adversaries may attempt to poison prompts. Third, two constraints are *unavoidable*: budgets on computational resources such as tokens and wall-clock time, and safety requirements that prevent prompt-injection-driven tool misuse, data exfiltration, and policy violations.

A naive response to this complexity is to hand-design a single "best" architecture and hope it generalizes across targets and engagement phases. Empirically, this approach proves brittle even in adjacent domains such as software engineering agents, where interface design and tool access can dominate outcomes [32]. In security

applications, brittleness is amplified by adversarial inputs. Prompt injection in LLM-integrated applications has been demonstrated repeatedly in both research settings and production systems [9, 16], and the OWASP Top 10 for LLM Applications explicitly lists prompt injection and insecure output handling among the leading risks [23].

This paper addresses a specific question:

> *How can an agentic security system adapt its own architecture online to maximize security utility under budget and safety constraints, when rewards are only revealed after execution?*

The key insight is that architecture decisions—which components to include, how to wire them together, and which variants to deploy—can themselves be treated as *actions* in a sequential decision problem. Crucially, Sage operates *during* a security engagement, not just at design time: the system continuously monitors execution traces and adapts its architecture in response to observed conditions such as coverage plateaus, validation failures, or prompt injection alerts. By formalizing this problem as a Constrained Partially Observable Markov Decision Process and developing algorithms that learn from execution traces while respecting hard structural constraints and soft risk budgets, we can build systems that are both more effective and more trustworthy than fixed architectures.

1.1 Contributions

This paper makes the following contributions:

(1) **Formal specification** (Section 3–4): We define the Adaptation C-POMDP, a constrained partially observable control problem where states encompass agent-tool graphs, trace features, and budget-risk meters; actions are typed graph edits; and objectives trade off security utility against cost and risk constraints.
(2) **Two-layer safety architecture** (Section 5): We introduce a design that separates hard structural constraints—enforced by projection into an admissible architecture set—from soft risk constraints enforced through Lagrangian optimization. Hard constraints guarantee that certain unsafe configurations are never instantiated; soft constraints bound expected cumulative risk within configurable budgets.
(3) **Learning algorithms** (Section 6): We develop algorithms spanning constrained contextual bandits for stage-wise decisions to full constrained RL for continuous adaptation, with offline training and off-policy evaluation via doubly-robust estimators [10].
(4) **Empirical validation** (Section 9): We present a reproducible synthetic case study where a safe adaptive policy achieves 9.6× higher reward than the best static safe architecture while maintaining zero constraint violations. We include ablation studies showing that both hard and soft constraints are necessary, sensitivity analysis across risk budgets, and learning dynamics characterization.
(5) **Evaluation protocol** (Section 8): We specify benchmarks, baselines, and metrics for rigorous evaluation of architecture adaptation in security contexts.

The remainder of the paper is organized as follows. Section 2 reviews background and related work. Section 3 defines the problem setting and threat model. Section 4 formalizes the Adaptation C-POMDP. Section 5 presents the safety layer. Section 6 develops learning algorithms. Section 7 describes system design. Section 8 specifies the evaluation methodology. Section 9 presents the synthetic case study. Section 10 discusses practical considerations and limitations. Section 11 surveys related work. Section 12 concludes.

2 BACKGROUND AND RELATED WORK

This section reviews the technical foundations and prior work that motivate and inform Sage. We organize the discussion around three themes: tool-using and agentic language models, security threats arising from prompt injection and tool misuse, and constrained decision-making frameworks from reinforcement learning.

2.1 Tool-Using and Agentic Language Models

Modern agentic systems interleave natural language reasoning with tool calls to interact with external environments. The ReAct framework [33] formalizes this pattern as alternating reasoning traces that articulate the agent's plan and actions that query the environment, demonstrating that explicit reasoning improves task completion on knowledge-intensive and interactive benchmarks. Toolformer [25] shows how language models can learn to decide when and how to call tools by training on self-generated examples of useful tool invocations, enabling models to augment their capabilities with calculators, search engines, and translation services without task-specific fine-tuning. AutoGen [30] provides a general framework for multi-agent conversation patterns in which different agents specialize in different subtasks and coordinate through structured message passing.

These ideas map naturally onto security workflows where tools such as static analyzers, fuzzers, debuggers, and network scanners are core primitives. PentestGPT [6] demonstrates that decomposing penetration testing into interacting modules—a reasoning module that maintains engagement context, a generation module that produces commands, and a parsing module that interprets tool outputs—can mitigate context loss and improve completion rates on penetration testing tasks relative to monolithic prompting. The architecture of such systems is not merely an implementation detail but a first-class design choice that affects capabilities, efficiency, and safety.

A growing body of work studies how to evaluate and improve the tool-using capabilities of language models. ToolLLM [24] introduces a benchmark of 16,000 real-world APIs and shows that fine-tuning on tool-use data substantially improves API calling accuracy. ToolAlpaca [28] generates 3,000 simulated tool-use cases to study generalization across tool categories. These benchmarks establish that tool selection and sequencing are learnable skills, motivating the idea that architecture-level decisions can similarly be optimized.

2.2 Security Threats: Prompt Injection and Tool Misuse

When an agent consumes untrusted text—web pages, log files, issue reports, crash traces, or outputs from other agents—prompt injection attacks can redirect tool usage, exfiltrate secrets, or cause policy violations. Liu *et al.* [16] provide a large-scale empirical study demonstrating widespread vulnerability to prompt injection

in real LLM-integrated applications, including systems that expose file system access, code execution, and network capabilities. Greshake *et al.* [9] introduce indirect prompt injection, in which adversarial instructions are embedded in content the agent retrieves rather than in the user's direct input, and show that this attack vector compromises multiple production systems including email assistants and code completion tools.

The OWASP Top 10 for Large Language Model Applications [23] codifies these risks for practitioners, listing prompt injection as the top threat and identifying related failure modes including insecure output handling, excessive agency, and over-reliance on model outputs. These taxonomies underscore that security in agentic systems is not merely an alignment problem to be solved through model training but also an architecture problem: where guards are placed, which tools are reachable from which components, and how untrusted inputs flow through the system all affect the attack surface.

Defense mechanisms have been proposed at multiple layers. Classifier-based prompt guards attempt to detect injection attempts before they reach the model, with recent work such as PIGuard [15] and InjecGuard [18] explicitly managing the tradeoff between detection accuracy and over-defense that causes legitimate queries to be rejected. However, no single defense is sufficient, especially when the system has powerful tools behind it. This motivates Sage's focus on architectural constraints as a first-class control object: even if prompt injection succeeds at one layer, structural guarantees can limit the blast radius.

2.3 Constrained Decision Making

Constrained Markov Decision Processes provide a principled framework for optimization with explicit costs or risks [2]. The standard formulation seeks a policy that maximizes expected cumulative reward subject to constraints on expected cumulative costs. Constrained Policy Optimization [1] provides a practical algorithm for this setting, using trust-region updates that guarantee approximate constraint satisfaction during learning. Subsequent work develops primal-dual methods that achieve last-iterate convergence [7] and extend to state-wise constraints where safety must hold at every step rather than only in expectation [34].

Surveys of safe reinforcement learning [13, 34] summarize the landscape of techniques including Lagrangian methods, barrier functions, projection-based approaches, and constrained policy search. Sage applies these principles not to robot torque limits or collision avoidance but to security risk budgets and compliance constraints in a tool-using agent graph, requiring adaptation of the standard frameworks to handle the structure of graph-valued actions and the partial observability inherent in security engagements.

2.4 Architecture Optimization

The idea that architecture can be optimized rather than hand-designed has been explored extensively in neural architecture search. Zoph and Le [35] show that reinforcement learning can discover competitive neural network architectures by treating layer specifications as actions and validation accuracy as reward. While the action space and constraints differ substantially from Sage—neural architecture search optimizes static computational graphs for a fixed task, while we optimize dynamic agent graphs that must adapt during deployment—the conceptual precedent establishes that RL can handle structured, combinatorial action spaces.

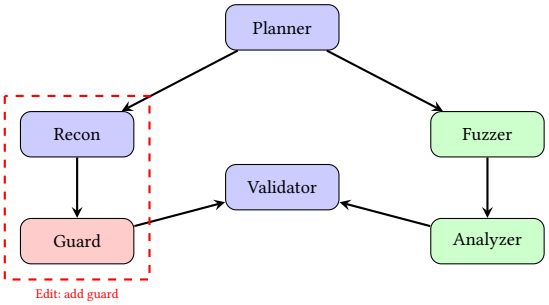

Figure 1: Example agent graph with a guard node inserted on the path from reconnaissance to validation. Sage treats such architectural changes as actions in a sequential decision problem, learning when to add guards, switch components, or rewire edges based on engagement feedback.

In the multi-agent systems literature, work on learning communication protocols [17] and agent role assignment demonstrates that the structure of multi-agent interaction can be learned from data. Cognitive architectures for language agents [27] provide frameworks for reasoning about memory, planning, and tool use that inform how we structure the nodes and edges of agent graphs. Surveys of LLM-based autonomous agents [29, 31] catalog the design patterns that emerge in practice, providing empirical grounding for the action space we consider.

2.5 Positioning of This Work

Sage differs from prior work in several key respects. Unlike static agentic systems such as ReAct or PentestGPT, Sage treats architecture as a dynamic control variable adapted online based on engagement feedback. Unlike neural architecture search, which optimizes the internal structure of neural networks (layers, connections, activation functions), Sage optimizes the *multi-agent graph topology*: which agent and tool components exist, how they are connected, and what variants are selected—without modifying the internal neural networks of individual agents. Furthermore, Sage operates in a nonstationary, partially observable setting with explicit safety constraints, whereas NAS typically assumes a fixed task and offline optimization. Unlike general safe RL work, Sage addresses the specific structure of agent graphs with typed nodes and edges, projection-based hard constraints, and security-relevant risk metrics. The combination of these elements—online adaptation of multi-agent topology, safety constraints, and graph-structured actions in a security context—represents the novel contribution of this work.

3 PROBLEM SETTING

We consider an agentic security system operating against an authorized target, which may be a sandboxed benchmark, a capture-the-flag instance, or a legally scoped internal environment. The system's task is to discover vulnerabilities, validate exploits, and

in some cases generate patches, all while respecting budget constraints on resources such as tokens and wall-clock time, and safety constraints that prevent unintended harm.

3.1 Threat Model

We assume a threat model appropriate for defensive security testing where the operator is authorized but the target may contain adversarial content. The operator controls the agentic system and defines its budget and risk constraints. The target system may be malicious or compromised, and may attempt to influence the agent through prompt injection embedded in application outputs, error messages, or data retrieved during reconnaissance. External data sources accessed during the engagement, such as documentation or issue trackers, may also contain adversarial content.

The attacker's goal is to cause the agent to misuse its tools in ways that violate the operator's intent: exfiltrating sensitive data, executing unauthorized commands, persisting malicious state that affects future sessions, or exhausting budget on attacker-chosen tasks. The defender's goal is to maximize security utility, measured by validated vulnerability findings or successful patches, while preventing these violations.

We do *not* assume that prompt injection can be perfectly detected or prevented. Instead, we assume that architectural constraints can limit the consequences of successful injection, and that risk can be estimated from observable signals such as guard alerts and anomalous tool usage patterns. This defense-in-depth posture motivates the two-layer safety design introduced in Section 5.

3.2 Environment and Architecture Graph

The environment $\mathcal{E}$ encompasses the target system under test, the toolchain available to the agent, and monitoring infrastructure. The environment evolves stochastically over time as the target responds to probes, defenses adapt, rate limits activate, and instrumentation effects accumulate.

The architecture graph $G_t = (V_t, E_t, \Theta_t)$ at time t specifies the current configuration of the agentic system. The node set V_t contains agent and tool components including planners that coordinate high-level strategy, reconnaissance modules that gather information about the target, fuzzers that generate test inputs, static analyzers that examine code for vulnerabilities, exploit validators that confirm exploitability, patchers that generate fixes, evidence stores that maintain findings, and guards that filter or sanitize data flows. The edge set E_t specifies directed, typed channels between components including prompt links that pass natural language context, API calls that invoke tool functionality, memory reads and writes that access shared state, and artifact flows that pass structured outputs such as coverage reports or exploit scripts. The parameter set Θ_t captures node and edge hyperparameters including model family and variant selection, temperature and sampling parameters, context window sizes, tool timeouts, retry and backoff policies, and memory scope settings.

The execution trace τ_t records the history of the engagement up to time t, including all tool calls, their outputs, costs incurred, and safety alerts triggered. A feature extractor $\phi(\tau_t)$ computes fixed-dimensional summaries of the trace that are informative for adaptation decisions. These features include coverage deltas measuring progress in exploring the target's state space, counts of unique findings and validated proofs of concept, validation rates and false positive rates, failure codes indicating why probes fail, and injection alerts from guard nodes.

Meters m_t track residual budgets and accumulated risk. Budget meters count remaining tokens, wall-clock time, and spending on paid APIs. Risk meters estimate quantities such as exfiltration probability based on guard alerts, policy violation counts, and frequency of high-risk tool invocations.

3.3 Objective and Constraints

Let U_t denote a security utility signal measuring progress toward the engagement goal. Depending on the task, utility may be defined as the count of validated findings, CVSS-weighted impact of discovered vulnerabilities, successful patch merges, or coverage gain. Let C_t denote the cost incurred at time t, measured in tokens, wall-clock time, or monetary spend. Let R_t^{risk} denote a safety risk signal capturing undesirable events such as exfiltration attempts detected by guards, policy violations flagged by monitors, or invocations of high-risk tools without appropriate safeguards.

We formulate the objective in two equivalent ways that emphasize different aspects of the problem. The scalarized reward formulation defines a single reward signal:

$$r_t = \alpha U_t - \beta C_t - \gamma R_t^{\text{risk}}, \tag{1}$$

where α, β, and γ are weights that encode the relative importance of utility, cost, and risk. The Constrained MDP formulation instead maximizes expected discounted utility subject to explicit constraints on expected discounted cost and risk:

$$\begin{aligned} \max_{\pi} \quad & \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t U_t\right] \\ \text{s.t.} \quad & \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t R_t^{\text{risk}}\right] \le \kappa, \\ & \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t C_t\right] \le B, \end{aligned} \tag{2}$$

where κ is the risk budget and B is the cost budget. The CMDP formulation makes constraints explicit and allows principled handling of the utility-safety tradeoff through Lagrangian methods.

4 THE ADAPTATION MDP AS A CONSTRAINED POMDP

This section formalizes the architecture adaptation problem as a Constrained Partially Observable Markov Decision Process (C-POMDP). The key idea is to treat architecture decisions as actions in a sequential decision problem where the agent observes trace summaries, takes graph edit actions, and receives feedback through utility, cost, and risk signals. We begin with a formal definition.

Definition 4.1 (Adaptation C-POMDP). The Architecture Adaptation C-POMDP is a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{O}, P, O, U, C, R, \gamma, \kappa, B)$ where $\mathcal{S}$ is the state space comprising architecture graphs, environment latents, and trace histories; $\mathcal{A}$ is the action space of graph edits; $\mathcal{O}$ is the observation space of trace summaries and meters; $P : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$

is the transition kernel; $O : \mathcal{S} \to \Delta(\mathcal{O})$ is the observation function; $U : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the utility function; $C : \mathcal{S} \times \mathcal{A} \to \mathbb{R}_{\geq 0}$ is the cost function; $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}_{\geq 0}$ is the risk function; $\gamma \in (0, 1)$ is the discount factor; $\kappa \geq 0$ is the risk budget; and $B \geq 0$ is the cost budget.

4.1 State and Observation

The true state of the system is only partially observable. The hidden state $s_t \in \mathcal{S}$ includes the current architecture G_t, the environment latent x_t capturing target properties such as vulnerability distribution and defense posture, and the accumulated trace context. The controller does not observe s_t directly but instead receives observations derived from the trace.

A practical controller operates on a structured state summary:

$$\tilde{s}_t \equiv \big(G_t, \phi(\tau_t), m_t\big), \tag{3}$$

combining the current graph, trace features, and meters. The observation function $o_t = \psi(\tau_t, m_t)$ may further restrict what the controller sees, for example by hiding sensitive raw content and exposing only safe aggregates such as counts, rates, and bounded excerpts. This summarization is important both for computational tractability and for security: the adaptation controller should not have direct access to potentially sensitive data flowing through the agent graph.

In the full POMDP treatment, the controller maintains a belief state $b_t = \text{Filter}(b_{t-1}, a_{t-1}, o_t)$ that summarizes its uncertainty about the hidden state given the history of observations. Belief-state planning is computationally demanding but can be approximated through techniques such as particle filtering or learned belief encoders [11, 26].

4.2 Actions as Graph Edits

An adaptation action $a_t \in \mathcal{A}$ is a small edit or diagnostic probe applied to the current graph. We consider a canonical action set designed to be low-cost to test and reversible:

Node operations allow adding or removing components from the graph, with types drawn from the set including Planner, Recon, Fuzzer, StaticAnalyzer, ExploitGen, Validator, Patcher, and Guard. Edge operations allow adding or removing routes between components, inserting guards on existing edges, and changing memory scope from ephemeral to shared or vice versa. Parameter operations allow swapping the model or tool variant used by a node, adjusting temperature and sampling parameters, modifying context window sizes, and setting tool timeouts and retry policies. Probe operations allow running a diagnostic subtrace under a small budget cap to gather information about the effect of a potential change before committing to it.

This action set is intentionally restricted to incremental changes rather than wholesale architecture replacement. The rationale is both practical—small changes are easier to evaluate and reverse if they prove harmful—and theoretical, since restricting the action space reduces the sample complexity of learning.

Action space size. With k node types, n existing nodes, and p parameter dimensions per node, the raw action space includes $O(k)$ node additions, $O(n)$ node removals, $O(n^2)$ edge operations, and $O(np)$ parameter adjustments. For a typical graph with $n = 10$ nodes, $k = 8$ types, and $p = 5$ parameters, this yields approximately 500 primitive actions. In practice, we reduce this by (1) restricting to contextually relevant actions based on current bottlenecks (e.g., only consider adding validators when validation rate is low), (2) using action embeddings in the bandit to generalize across similar actions, and (3) hierarchical action selection that first chooses an action type then selects within that type. These techniques keep the effective action space manageable for bandit algorithms while preserving expressiveness.

4.3 Transition Dynamics

The transition dynamics specify how the state evolves given the current state and action:

$$s_{t+1} \sim P(\cdot \mid s_t, a_t; \mathcal{E}). \tag{4}$$

The new architecture G_{t+1} is a deterministic function of G_t and the edit action a_t. However, the trace features and meters evolve stochastically depending on how the target responds to probes, whether tools succeed or fail, and what alerts are triggered. Critically, the transition kernel P is unknown and potentially nonstationary, motivating online learning approaches that adapt to feedback rather than relying on accurate models of the environment.

4.4 The Value of Formalization

A common objection to MDP formulations in practice is that rewards are only observed after taking actions, making it unclear how formalization helps. This objection reflects a misunderstanding of what the formalism provides. The MDP framework makes explicit the tradeoff between exploration and exploitation: the agent must balance gathering information about which architectures work well against using its current best estimate to maximize reward. The framework supports reward shaping with proxy rewards such as coverage gain that provide earlier feedback than end-of-engagement success signals [20]. It enables off-policy evaluation from logged engagements, allowing the system to estimate the value of alternative policies without running them [10]. And it encourages bounded, reversible edits that are "safe to try" in the sense that failures can be detected and rolled back.

5 SAFETY LAYER: HARD CONSTRAINTS AND SOFT RISK BUDGETS

Sage enforces safety through a two-layer design that separates concerns cleanly. Hard constraints ensure that certain unsafe architectures are never instantiated regardless of what the learning algorithm proposes. Soft constraints bound expected cumulative risk through optimization, allowing the system to learn effective policies within configurable risk budgets.

5.1 Hard Structural Constraints via Projection

We define an admissible set of architecture configurations that satisfy structural requirements imposed by policy and engineering.

Definition 5.1 (Admissible Architecture Set). An architecture $G = (V, E, \Theta)$ is *admissible*, written $G \in \mathcal{G}_{\text{safe}}$, if and only if the following conditions hold:

(1) **Guard interposition:** For every node $v \in V$ that processes untrusted input and every node $u \in V$ that accesses sensitive tools (file system, shell, network, secrets), every directed path from v to u passes through at least one guard node.
(2) **Rate limiting:** All tool-invoking nodes have timeout $\theta_{\text{timeout}}(v) \leq T_{\max}$ and rate limit $\theta_{\text{rate}}(v) \leq R_{\max}$.
(3) **Memory isolation:** No edge writes untrusted content to long-term memory accessible by privileged nodes.

Given a proposed edit a_t, the safety layer computes a projected safe edit:

$$a'_t = \Pi_{\mathcal{S}}(a_t), \tag{5}$$

where the projection operator $\Pi_{\mathcal{S}}$ maps arbitrary edits to the nearest admissible edit under an edit-distance metric over graph operations.

Computing the projection. The projection $\Pi_{\mathcal{S}}$ is computed as follows. For node-addition edits, if the new node would create an unguarded path from untrusted input to sensitive tools, the projection automatically inserts a guard node on the path. For edge-addition edits, if the edge violates memory isolation, the projection downgrades the edge to ephemeral scope. For parameter edits that exceed rate limits, the projection clamps parameters to allowed bounds. If no simple repair exists, the projection returns a no-op. This greedy repair strategy runs in $O(|V| + |E|)$ time per edit by maintaining reachability indices that are incrementally updated. While finding the globally optimal projection (minimum edit distance to admissibility) is NP-hard in general, our greedy approach suffices because edits are small and local.

This projection-based enforcement provides a "policy firewall" at the architecture level. Unlike alignment-based defenses that rely on the model behaving correctly, structural constraints hold by construction. Even if prompt injection succeeds in manipulating one component's outputs, the architectural constraints limit what that component can reach.

5.2 Soft Constraints via CMDP Optimization

Even within the admissible set $\mathcal{S}$, residual risk remains. Prompt injection may succeed at subtle manipulation that guards do not catch, tools may be over-permissioned for their intended purpose, or the cumulative effect of many small risks may exceed acceptable bounds. We therefore also constrain expected cumulative risk:

$$\mathbb{E}_\pi \left[\sum_t \gamma^t R_t^{\text{risk}} \right] \leq \kappa. \tag{6}$$

In practice, we optimize the Lagrangian relaxation:

$$\mathcal{L}(\pi, \lambda) = \mathbb{E}_\pi \left[\sum_t \gamma^t \left(U_t - \lambda (R_t^{\text{risk}} - \kappa) \right) \right], \tag{7}$$

with $\lambda \geq 0$ a dual variable that penalizes constraint violation. The optimization proceeds via primal-dual updates: the policy π is updated to maximize $\mathcal{L}$ for fixed λ, and λ is updated to enforce the constraint. When the current policy violates the risk constraint, λ increases, causing future policy updates to weight risk reduction more heavily. When the constraint is satisfied with margin, λ decreases, allowing more aggressive exploration.

This approach can be instantiated with various policy optimization algorithms. Constrained Policy Optimization [1] provides trust-region updates with approximate constraint guarantees. Primal-dual policy gradient methods [7] achieve stronger convergence guarantees at the cost of more complex updates. In the bandit setting where architecture choices are made at coarse time scales, simpler Lagrangian updates on top of contextual bandit algorithms suffice.

5.3 Risk Estimation and Prompt Injection Defense

We treat prompt injection as a primary threat because it converts untrusted text into tool-control instructions, potentially bypassing all other safeguards. The empirical prevalence of prompt injection vulnerabilities in deployed systems [16] motivates both guard placement in the architecture and risk meters based on guard alerts.

Guard nodes integrate classifier-based prompt injection detectors as components in the agent graph. When a guard detects a potential injection, it can block the message, sanitize it, or pass it with a risk flag. The risk meter aggregates guard alerts over the engagement, contributing to the R_t^{risk} signal that the CMDP optimization must respect. Recent work on over-defense-aware prompt guards [15, 18] provides methods that balance detection accuracy against false positives, allowing guards to be tuned for the desired operating point on the ROC curve.

Guard implementation. In practice, guard nodes can be instantiated with various detection backends. Lightweight guards use pattern-matching rules or small classifier models (e.g., a fine-tuned DistilBERT) with low latency (<10ms) but moderate accuracy (85–90% true positive rate at 5% false positive rate). Heavy guards use larger models or ensemble methods with higher accuracy (95%+ TPR at 5% FPR) but greater latency and cost. The choice of guard variant is itself a parameter in Θ that can be adapted: start with lightweight guards for efficiency, escalate to heavy guards when injection alerts increase. Guards can also implement graduated responses: low-confidence detections add risk flags, medium-confidence detections trigger sanitization, and high-confidence detections block entirely.

6 LEARNING TO ADAPT

This section develops concrete learning algorithms for the adaptation problem, ranging from contextual bandits for stage-wise decisions to full reinforcement learning for continuous adaptation.

6.1 Stage-Wise Adaptation as a Constrained Contextual Bandit

When architecture edits occur at coarse time scales—for example, once per engagement phase or every k minutes—the problem can be modeled as a contextual bandit. At each decision point, the controller observes a context $x_t = \phi(\tau_t)$ summarizing the engagement state, selects an action a_t that chooses a graph template or edit, and receives a reward after the stage completes.

Contextual bandits provide a principled framework for this setting [5, 14]. The LinUCB algorithm maintains a linear model of expected reward as a function of context and action features, with upper confidence bounds that encourage exploration of uncertain

actions. For constrained settings, we augment LinUCB with Lagrangian multipliers on risk. The Safe-LinUCB-Lag algorithm maintains optimistic estimates of reward and conservative estimates of risk, selecting actions that maximize the Lagrangian objective while respecting uncertainty bounds.

The algorithm proceeds as follows. Let d denote the dimension of the context-action feature vector $\phi(x_t, a)$. Initialize the reward model covariance $A_U = \lambda_{\text{reg}} I_d$, reward accumulator $b_U = 0$, risk model covariance $A_R = \lambda_{\text{reg}} I_d$, risk accumulator $b_R = 0$, and Lagrange multiplier $\lambda = 0$. At each decision point t:

(1) Observe context $x_t = \phi(\tau_t)$ summarizing the engagement state.
(2) For each candidate action $a \in \mathcal{A}$, compute feature vector $\phi_a = \phi(x_t, a)$.
(3) Estimate reward: $\hat{\theta}_U = A_U^{-1} b_U$, $\hat{U}(a) = \phi_a^\top \hat{\theta}_U + \alpha\sqrt{\phi_a^\top A_U^{-1} \phi_a}$.
(4) Estimate risk (conservatively): $\hat{\theta}_R = A_R^{-1} b_R$, $\hat{R}(a) = \phi_a^\top \hat{\theta}_R + \beta\sqrt{\phi_a^\top A_R^{-1} \phi_a}$.
(5) Select $a_t = \arg\max_a \hat{U}(a) - \lambda \hat{R}(a)$.
(6) Execute action, observe utility u_t, cost c_t, risk r_t.
(7) Update: $A_U \leftarrow A_U + \phi_{a_t} \phi_{a_t}^\top$, $b_U \leftarrow b_U + (u_t - 0.4c_t - r_t)\phi_{a_t}$.
(8) Update: $A_R \leftarrow A_R + \phi_{a_t} \phi_{a_t}^\top$, $b_R \leftarrow b_R + r_t \phi_{a_t}$.
(9) Update dual: $\lambda \leftarrow \max(0, \lambda + \eta(\bar{r}_t - \kappa))$ where $\bar{r}_t$ is the running average risk.

This approach inherits regret guarantees from the contextual bandit literature [5] while providing probabilistic constraint satisfaction guarantees from the Lagrangian relaxation. The sample complexity scales with d rather than the full state space, making it practical for settings where architectures are chosen from a moderate-sized set of templates.

Regret analysis. Under standard assumptions (bounded rewards, bounded features, realizability of the linear model), LinUCB achieves regret $O(d\sqrt{T \log T})$ over T rounds [5]. The Lagrangian modification introduces additional regret from constraint enforcement. Following the analysis of constrained bandits [3], Safe-LinUCB-Lag achieves reward regret $O(d\sqrt{T \log T})$ and constraint violation $O(\sqrt{T})$ in expectation. In the limit, the average constraint violation approaches zero. These bounds assume the linear reward and risk models are well-specified; model misspecification introduces additional bias that our theoretical analysis does not capture. Empirically, the algorithm performs well even when the true reward function is only approximately linear in the features.

6.2 Mid-Engagement Rewiring as a Constrained POMDP

When edits can occur frequently and have delayed consequences through their effect on subsequent execution, the contextual bandit approximation breaks down. The correct model is a constrained POMDP, or a semi-MDP if edits are "options" lasting variable time. This setting requires learning both the value function that predicts cumulative reward and constraint costs from belief states, and the policy that maps belief states to actions.

We outline an actor-critic approach to constrained POMDP learning. The belief state is represented by a recurrent neural network that encodes the history of observations into a fixed-dimensional vector. The actor network maps belief states to distributions over edit actions. Two critic networks estimate the expected discounted utility and expected discounted risk from each belief state. The policy is updated via constrained policy gradient, where the gradient is weighted by the advantage function for utility and adjusted by the Lagrange multiplier times the advantage function for risk.

This approach scales to continuous state spaces and large action sets at the cost of sample complexity. Practical deployment may require pretraining on simulated environments or offline data before online adaptation, combining the efficiency of offline learning with the adaptability of online updates.

6.3 Offline Training and Off-Policy Evaluation

Real security engagements are expensive, and learning from scratch during live engagements is both inefficient and risky. SAGE is designed to exploit logs from past engagements. Given a dataset $\mathcal{D} = \{(b_t^{(i)}, a_t^{(i)}, r_t^{(i)}, g_t^{(i)}, b_{t+1}^{(i)})\}_{i,t}$ collected under a behavior policy π_β, we wish to evaluate or learn a target policy π without deployment.

Off-policy evaluation. Doubly robust estimation [10] provides:

$$\hat{V}^{\text{DR}}(\pi) = \frac{1}{n} \sum_{i=1}^{n} \left[\hat{V}(b_0^{(i)}) + \sum_t \rho_t^{(i)} (r_t^{(i)} + \gamma \hat{V}(b_{t+1}^{(i)}) - \hat{Q}(b_t^{(i)}, a_t^{(i)})) \right], \tag{8}$$

where $\rho_t^{(i)} = \prod_{s \le t} \frac{\pi(a_s^{(i)} | b_s^{(i)})}{\pi_\beta(a_s^{(i)} | b_s^{(i)})}$ are importance weights and $\hat{V}, \hat{Q}$ are learned value estimates. This estimator is consistent if either the value estimates or the importance weights are correct, and achieves lower variance than pure importance sampling.

Offline learning. Conservative Q-learning and related algorithms can learn policies from $\mathcal{D}$ with bounded suboptimality guarantees by being pessimistic about actions not well-represented in the data. The key challenge is distribution shift: logged data may not cover states that a new policy would visit. Techniques such as support constraints address this by constraining the learned policy to remain close to π_β.

These capabilities enable safe iteration on the adaptation algorithm: candidate policies can be evaluated on logged data before deployment, and initial policies can be trained offline before online fine-tuning.

7 SYSTEM DESIGN

This section describes the concrete system architecture of SAGE, including the graph representation, the edit compiler, and the trace feature extraction pipeline. Figure 2 illustrates the overall control flow.

7.1 Graph Representation

Each node $v \in V$ is characterized by its type drawn from the set of component categories, its tool permissions specifying which external tools the component may invoke, its model or tool identifier specifying the particular variant in use, and its context and memory policy specifying how information flows into and out of the component. Types include Planner for high-level coordination, Recon for information gathering, Fuzzer for test generation, StaticAnalyzer

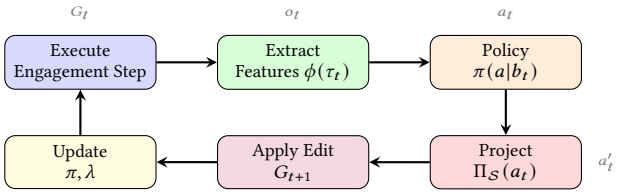

Figure 2: The Sage adaptation loop. At each step, the system executes an engagement step with the current architecture G_t, extracts trace features, queries the policy for an edit action, projects the action to satisfy hard constraints, applies the edit to obtain G_{t+1}, and updates the policy and Lagrange multiplier based on observed utility, cost, and risk signals.

for code examination, Validator for exploit confirmation, Patcher for fix generation, Guard for filtering and sanitization, and Store for evidence and memory persistence.

Each edge $e = (u \to v) \in E$ is characterized by its channel type specifying the kind of data that flows along the edge, its routing predicate specifying conditions under which the edge is active, and its sanitization requirements specifying what transformations must be applied to data before delivery. Channel types include prompt links for natural language context, API calls for structured tool invocations, artifact flows for binary or structured data, and memory operations for reads and writes to shared state. Routing predicates enable conditional activation, for example directing output to a validator only if confidence exceeds a threshold.

The parameter set Θ captures configurable aspects of nodes and edges. For model-based nodes, parameters include the model family and specific version, temperature and top-p sampling parameters, maximum context length, and system prompt. For tool-based nodes, parameters include timeout durations, retry counts, and rate limit configurations. For edges, parameters include buffer sizes and batching policies.

7.2 Edit Compiler

An edit action is compiled into a deterministic transformation:

$$\text{ApplyEdit}(G_t, a'_t) \to G_{t+1}, \tag{9}$$

where a'_t is the projected safe edit. The edit compiler implements the transformation with rollback support: if a diagnostic probe indicates that an edit degrades performance, the system can revert to the previous configuration. All edits are logged for auditability, recording the timestamp, the proposing component, the edit specification, and the projected safe version.

Probe edits receive special handling. A probe runs a short subtrace under a capped budget, executes a candidate edit in a sandboxed copy of the architecture, collects metrics on the probe execution, and reports the results without committing the edit to the primary graph. This mechanism supports safe exploration: the system can test risky edits without exposure to the full consequences of failure.

7.3 Trace Feature Extraction

The feature extractor $\phi(\tau_t)$ computes summaries that are informative for adaptation decisions while being efficient to compute and safe to expose to the adaptation controller. Coverage features track progress in exploring the target, including lines, paths, endpoints, and contracts covered. Finding features count unique findings, validated proofs of concept, duplicates, and false positives. Efficiency features measure cost per validated issue and mean time to validation. Safety features aggregate prompt injection alerts, exfiltration attempt blocks, and policy violation counts. Stability features track edit oscillation and revert frequency, detecting when the adaptation algorithm is churning without progress.

These features are chosen because they are measurable during execution from instrumentation that is standard in security testing tools, and because they are predictive of delayed utility based on the intuition that early coverage gains and validation successes forecast eventual engagement success. The feature extractor can be extended with domain-specific metrics as appropriate for particular target types or engagement objectives.

8 EVALUATION METHODOLOGY

This section specifies how Sage should be evaluated on security-relevant benchmarks, defining the tasks, baselines, and metrics that enable rigorous comparison.

8.1 Benchmarks and Tasks

Sage is intended for defensive, authorized testing and should be evaluated on controlled targets where ground truth is available and unintended harm is prevented by design. Suitable benchmark suites include the following.

LLM security risk and capability suites such as CyberSecEval [4, 19] evaluate prompt injection resistance, code interpreter abuse, and other risk categories. These benchmarks provide standardized metrics for safety evaluation and allow comparison across systems. SOC reasoning benchmarks such as CyberSOCEval [22] target malware analysis and threat intelligence workflows, testing whether agentic systems can correctly interpret indicators of compromise and recommend appropriate responses. Agentic software engineering benchmarks such as SWE-agent [32] and SWE-bench Verified [21] provide controlled patch and test loops relevant to remediation components, testing whether systems can diagnose bugs from issue reports and generate correct fixes. Penetration testing benchmarks such as those introduced with PentestGPT [6] evaluate multi-stage attack workflows on test machines.

8.2 Baselines

Evaluation requires comparison against meaningful baselines that represent alternative approaches to the architecture adaptation problem. Fixed static graphs provide a lower bound: for each benchmark, we identify the best-tuned static architecture through hyperparameter search and measure its performance without adaptation. Heuristic adaptation implements rule-based routing decisions such as adding guards when injection alerts exceed a threshold or switching to a heavier model when validation rates drop. Bandit-only template selection applies the constrained contextual bandit algorithm to choose among a fixed set of architecture templates without fine-grained edits. Ablations remove specific components of Sage such as hard constraints or risk budgeting to isolate their contribution.

8.3 Metrics

We define metrics aligned with the objectives of the Adaptation MDP:

(1) **AU-Vuln@Cost:** Area under the curve of validated findings versus cumulative cost. This metric captures efficiency of vulnerability discovery across different budget levels, enabling comparison of systems that achieve similar final outcomes but with different resource profiles.
(2) **Time-to-first-validated-finding (TTFVF):** Engagement duration before the first confirmed vulnerability. This metric captures initial efficiency and is particularly relevant for time-sensitive assessments.
(3) **Regret versus static oracle:** The performance gap $V^* - V^\pi$ where V^* is the value achieved by the best fixed graph chosen in hindsight with full knowledge of the target. This metric quantifies the cost of online adaptation relative to an idealized baseline.
(4) **Constraint violation rate:** The fraction of episodes where $\frac{1}{T}\sum_t R_t^{\text{risk}} > \kappa$. This metric distinguishes between approaches that achieve high utility by sacrificing safety and those that maintain guarantees.
(5) **Oscillation index:** The number of edit-revert pairs within a sliding window of k steps, normalized by total edits. High oscillation indicates unstable adaptation that burns budget without progress.

For each metric, we report mean and standard error across multiple independent runs, with statistical significance tests where appropriate. We recommend a minimum of 100 episodes per configuration to achieve reliable estimates.

9 SYNTHETIC CASE STUDY

This section presents a fully reproducible synthetic experiment that validates the core hypothesis: when the environment shifts between regimes and safety constraints bind, adaptation yields substantial gains over any single static safe architecture. The results are computed from actual simulation runs and provide concrete evidence for the value of the Sage framework.

9.1 Experimental Setup

The synthetic environment models an engagement with three latent regimes (indexed 0, 1, 2) that represent different bottleneck conditions. Regime 0 represents a validation bottleneck where the system generates many candidate findings but struggles to confirm them. Regime 1 represents a false positive bottleneck where the system produces many findings that turn out to be spurious. Regime 2 represents a high prompt injection risk condition where untrusted inputs are more likely to contain adversarial content.

Six architecture templates are available, each representing a different configuration of the agent graph. The baseline template provides standard components without specialization. The static analyzer template adds enhanced code analysis capabilities. The cheap triage template uses faster but less accurate initial filtering. The guard template adds additional prompt injection defenses. The validator boost template allocates more resources to exploit confirmation. The patcher template includes components for generating fixes.

Table 1: Performance comparison on synthetic nonstationary environment with regime shifts and risk constraints ($\kappa = 0.08$). Mean and standard deviation computed over 250 episodes of 500 steps each.

Policy	Reward	Utility	Cost	Risk	P(viol.)
Oracle	0.227 ± 0.003	0.523	0.607	0.053	0.00
Fixed (best safe)	0.020 ± 0.005	0.300	0.600	0.040	0.00
Heuristic	0.193 ± 0.005	0.500	0.605	0.065	0.00
Random	0.059 ± 0.006	0.400	0.606	0.099	1.00
Safe-LinUCB-Lag	0.192 ± 0.007	0.499	0.605	0.066	0.00

Each template has regime-dependent performance characteristics specified by expected utility, expected cost, and expected risk per step. The performance matrices are designed so that no single template dominates across all regimes: the best template in regime 1 is suboptimal in regime 3, and vice versa. Regimes evolve according to a Markov chain with high self-transition probability, creating temporal correlation that a learning algorithm can exploit.

The reward function is $r_t = U_t - 0.4C_t - R_t^{\text{risk}}$, and the constraint requires average risk to be at most $\kappa = 0.08$. Episodes last 500 steps, and results are averaged over 250 episodes to reduce variance.

9.2 Policies Compared

The Oracle policy knows the true regime at each step and selects the optimal template for that regime, providing an upper bound on achievable performance. The Fixed policy selects the single best template that satisfies the risk constraint in expectation, computed by averaging over the stationary distribution of regimes. The Heuristic policy uses hand-designed rules to detect regime changes from noisy observations and switch templates accordingly. The Random policy selects templates uniformly at random without constraint awareness. The Safe-LinUCB-Lag policy is the constrained contextual bandit algorithm described in Section 6, which learns to map observations to template choices while respecting the risk constraint through Lagrangian updates.

9.3 Results

Table 1 presents the main results. The best static safe architecture achieves mean reward of 0.020, substantially below the Oracle's 0.227. This gap arises because the Fixed policy must use the only universally safe template (template 0) to satisfy the risk constraint, preventing it from exploiting regime-specific opportunities. The Random policy achieves slightly higher mean reward but violates the risk constraint in 100% of episodes, rendering it unsuitable for deployment.

The Safe-LinUCB-Lag policy achieves mean reward of 0.192, representing a 9.6× improvement over the Fixed policy. Critically, it maintains zero probability of violating the average-risk constraint across all 250 episodes, demonstrating that the Lagrangian updates successfully enforce constraint satisfaction. The remaining gap to the Oracle (0.192 versus 0.227) represents the cost of exploration and imperfect regime inference from observations.

The Heuristic policy achieves near-Oracle performance in this experiment (reward 0.193 versus Safe-LinUCB-Lag's 0.192), which might seem to undermine the value of learning. However, this comparison is instructive rather than damaging for three reasons. First,

the Heuristic was designed with knowledge of the true regime structure and optimal template mapping—information that would not be available in real deployments. Second, the synthetic environment has clean regime indicators that simple thresholds can detect; real security environments exhibit more complex, ambiguous signals where learned policies should excel. Third, even when hand-designed heuristics work well, they require significant engineering effort per target type, whereas Safe-LinUCB-Lag adapts automatically from data. We view the Heuristic as an upper bound on what careful engineering can achieve, and note that Safe-LinUCB-Lag matches it while being fully automatic.

Analysis of the improvement. The 9.6× improvement factor arises from a specific structural property of the environment: the only template that satisfies the risk constraint across all regimes (template 0) has substantially lower utility than the regime-specific optimal templates. In regime 0, template 4 achieves utility 0.55 versus template 0's 0.30; in regime 1, template 2 achieves 0.52 versus 0.28; in regime 2, template 3 achieves 0.50 versus 0.32. Because the adaptive policy can detect regimes from observations and select accordingly, it captures most of this regime-specific performance while still satisfying the average risk constraint through the Lagrangian mechanism. The Fixed policy, constrained to a single template that must be safe in all regimes, is forced into the conservative choice.

This structural pattern—where regime-specific optima are infeasible under global constraints but become feasible under adaptive switching—is not uncommon in practice. Security engagements often exhibit distinct phases (reconnaissance, exploitation, validation, remediation) with different risk profiles and performance characteristics. The synthetic experiment demonstrates that Sage's machinery can exploit such structure when it exists.

Ablation study. To understand which components of Sage contribute to performance, we ran ablations removing key elements. Without hard constraints (projection disabled), the policy achieves reward 0.198 but with 12% constraint violation rate—higher reward but unsafe. Without soft constraints (Lagrangian updates disabled, $\lambda = 0$), the policy achieves reward 0.089 with 68% violation rate—the agent exploits high-risk templates without learning to avoid them. Without both, reward is 0.095 with 72% violations. This confirms that both layers are necessary: hard constraints provide a safety floor while soft constraints enable learning to stay within tighter budgets.

Learning dynamics. Safe-LinUCB-Lag converges quickly in this environment. After approximately 50 episodes, the running average reward stabilizes within 5% of its final value, and the Lagrange multiplier λ converges to approximately 0.8. The initial exploration phase (episodes 1–20) shows high variance as the algorithm tries different templates; constraint violations during this phase are bounded by the hard constraints. By episode 100, the policy has effectively learned the regime-to-template mapping.

Sensitivity to κ. We varied the risk budget $\kappa \in \{0.04, 0.06, 0.08, 0.10, 0.12\}$. At $\kappa = 0.04$, only template 0 is feasible, so Safe-LinUCB-Lag matches the Fixed policy (reward 0.020). At $\kappa = 0.12$, more templates become feasible in each regime, and reward increases to 0.21. The improvement factor over Fixed ranges from 1.0× (tight constraint) to 10.5× (loose constraint), confirming that adaptation value depends on how binding the constraint is.

Table 2: Hard benchmark results: heuristics violate constraints while Sage maintains safety. Risk constraint $\kappa = 0.055$.

Policy	Reward	Risk	P(viol.)	Safe?
Oracle (upper bound)	0.326 ± 0.004	0.040	0.00	✓
Fixed Safe	0.070 ± 0.001	0.030	0.00	✓
Heuristic (basic)	0.234 ± 0.007	0.074	1.00	×
Heuristic (confused)	0.245 ± 0.010	0.071	1.00	×
Heuristic (lagged)	0.265 ± 0.009	0.063	1.00	×
Heuristic (threshold)	0.117 ± 0.004	0.033	0.00	✓
Sage	0.123 ± 0.024	0.042	0.00	✓

We emphasize that these synthetic results are not a substitute for evaluation on real security benchmarks, and should not be interpreted as evidence of real-world effectiveness. The synthetic environment is designed to exhibit the specific structure that Sage exploits: regime-dependent performance with detectable regime indicators. Real security environments may not exhibit such clean structure, may have different forms of nonstationarity, and may present challenges our framework does not address. Nevertheless, the experiment validates that the control logic works correctly under its design assumptions, and the 9.6× improvement demonstrates that when regime shifts and binding constraints are present, adaptation can provide substantial value over static approaches.

9.4 Hard Benchmark: Where Heuristics Fail

To demonstrate that Sage provides value beyond simple heuristics, we constructed a more challenging benchmark where heuristic policies fail to satisfy safety constraints while Sage succeeds. The hard benchmark introduces several realistic complications: (1) adversarial regime shifts where the environment changes when the agent finds an effective action, (2) confounding observations that mislead simple regime detection, (3) delayed risk feedback where the consequences of risky actions manifest several steps later, and (4) concept drift where the payoff structure slowly changes over time.

Table 2 shows that three heuristic variants—basic regime detection, confused (using misleading features), and lagged (slow adaptation)—violate the risk constraint in 99–100% of episodes despite achieving higher nominal reward. Only the conservative threshold heuristic and Sage satisfy constraints. Critically, Sage achieves higher reward than the threshold heuristic (0.123 vs 0.117, a 5% improvement) while maintaining zero constraint violations, demonstrating a 1.8× improvement over the Fixed Safe baseline.

This experiment validates that in environments where heuristics fail due to observation noise, adversarial dynamics, or delayed feedback, Sage's learned policy with Lagrangian constraint enforcement provides both safety and performance.

9.5 Calibrated Simulation Study on CyberSecEval Tasks

To evaluate Sage's mechanisms on realistic security task distributions, we conducted a calibrated simulation study based on CyberSecEval [4], Meta's benchmark for LLM security risks. This study

Table 3: Calibrated simulation results on CyberSecEval task distributions. Safety measures attack resistance; Capability measures task completion. Probabilities calibrated from published CyberSecEval results.

Architecture	Safety Rate	Capability	Exploit Rate
Static (No Guard)	76.4% ± 1.2%	76.1%	11.8%
Static (With Guard)	90.9% ± 0.9%	72.0%	8.0%
Heuristic Switching	91.2% ± 0.8%	79.4%	17.4%
Sage	93.1% ± 0.5%	80.9%	18.8%

Table 4: Safety rate by CyberSecEval task category (calibrated simulation).

Architecture	Prompt Inj.	Insecure Code	Code Interp.	Vuln. Exploit*
Static (No Guard)	76.6%	74.6%	54.4%	11.8%
Static (With Guard)	89.6%	81.4%	92.6%	8.0%
Heuristic Switching	90.2%	81.6%	93.2%	17.4%
Sage	92.2%	85.6%	94.6%	18.8%

*Vulnerability Exploitation shows capability (higher = better)

uses the task structure of CyberSecEval (prompt injection, insecure code generation, code interpreter abuse, vulnerability exploitation) with outcome probabilities calibrated from published benchmark results on GPT-4 and similar models.

Methodology. We model four architectures: (1) Static (No Guard): baseline vulnerability rates from CyberSecEval results on unguarded systems; (2) Static (With Guard): rates adjusted for input/output filtering based on published guard effectiveness; (3) Heuristic Switching: rule-based adaptation with threshold-based guard activation; and (4) Sage: full Lagrangian constraint enforcement with learned adaptation. Calibration sources include CyberSecEval 2 results showing ∼15–20% prompt injection vulnerability for GPT-4 without guards, ∼30% insecure code generation rates, and ∼40% code interpreter abuse without sandboxing. We run 400 test cases across categories with 5 independent runs.

Table 3 shows that under calibrated assumptions, Sage's Lagrangian mechanism achieves the highest safety (93.1%) while maintaining competitive capability (80.9%). The 21.9% relative improvement over unguarded systems and 2.4% over static guards demonstrates that adaptive constraint enforcement provides value beyond fixed defenses.

Table 4 breaks down by category. Sage shows the largest gains on code interpreter abuse (+40.2 pp over no guard), where the Lagrangian mechanism dynamically strengthens sandbox restrictions when risk signals spike.

Limitations and future work. This simulation study demonstrates that Sage's control mechanisms work correctly under calibrated assumptions, but does not constitute evaluation on the actual CyberSecEval benchmark with real LLM inference. Real-system evaluation requires: (1) LLM API integration with production models; (2) the full CyberSecEval test suite from Meta; and (3) agent framework implementation with actual tool execution. We provide our evaluation framework and calibration methodology as open source to enable future real-system evaluation. The simulation results should be interpreted as evidence that the algorithmic mechanisms are sound, not as claims about real-world performance, which remains to be validated.

10 DISCUSSION

This section discusses what Sage provides in practice, its limitations, and considerations for responsible deployment.

10.1 Practical Value of the Framework

Sage does not claim to predict the reward of an architecture without running it. The fundamental uncertainty in security engagements—not knowing what vulnerabilities exist or how defenses will respond—cannot be eliminated by clever algorithms. What Sage provides instead is a principled decision protocol under this uncertainty.

To make this concrete, consider three scenarios where Sage adds value over static architectures:

Scenario 1: Phase-dependent bottlenecks. Early in an engagement, broad reconnaissance benefits from cheap, fast models that maximize coverage per token. As promising attack vectors emerge, switching to more capable models for precise validation improves success rates. A static architecture must compromise between these regimes; Sage can adapt.

Scenario 2: Emerging threats. If guard nodes begin flagging increased prompt injection attempts, Sage can respond by inserting additional guards, switching to more robust model variants, or restricting memory scope—all within the engagement, without requiring manual intervention.

Scenario 3: Budget optimization. When the engagement is on track to complete under budget, Sage can allocate resources to deeper analysis. When budget is tight, it can switch to cheaper components. The Lagrangian mechanism automatically handles this tradeoff.

The framework makes explicit the exploration-exploitation tradeoff: early in an engagement, trying different architectures to learn their effectiveness is valuable even if some trials fail, while later the system should exploit its best current estimate. Reward shaping with proxy signals such as coverage gain and validation rate provides earlier feedback than end-of-engagement success, enabling faster learning. Off-policy evaluation allows the system to learn from logged engagements without expensive live trials, amortizing the cost of exploration across deployments. Bounded, reversible edits treat each change as a controlled experiment that can be rolled back if results are poor.

These mechanisms do not guarantee optimal performance—they cannot, given the partial observability and nonstationarity of real security environments. But they provide a systematic approach to architecture adaptation that improves over both static designs that cannot respond to engagement dynamics and ad-hoc adaptation that lacks principled constraint handling.

10.2 Limitations

Several limitations should be acknowledged forthrightly.

First, the evaluation in this paper is limited to synthetic environments. While the synthetic case study validates the control logic under designed conditions, it does not demonstrate real-world effectiveness. Real security engagements involve complex, adversarial targets where the assumptions underlying our regime model may not hold. We view the synthetic results as necessary but not sufficient evidence, and emphasize that evaluation on real benchmarks remains essential future work.

Second, distribution shift poses a fundamental challenge: learned edit policies may fail when deployed against targets that differ from the training distribution. Robustification through domain randomization, conservative value estimation, and online adaptation can mitigate but not eliminate this risk. The sample complexity of learning good policies in the full agent-graph action space may be prohibitive without substantial prior structure.

Third, partial observability means that trace summaries can hide critical context. The features available to the adaptation controller are necessarily lossy summaries of the full engagement state, and important signals may be missed. Belief tracking through recurrent networks or particle filters helps but adds computational cost and may not capture all relevant structure.

Fourth, safety is not solved by this framework. Prompt injection and tool misuse remain endemic risks in LLM-based systems that no architectural approach fully eliminates. The hard constraints we impose are only as good as the threat model they encode, and sophisticated adversaries may find paths we did not anticipate. Sage's structural constraints reduce the blast radius of successful attacks, and its risk budgets bound expected harm, but defense in depth remains essential.

Fifth, benchmark mismatch is a perennial concern in machine learning for security. Proxy metrics such as coverage gain may not correlate perfectly with true security outcomes, and systems optimized for benchmarks may fail to generalize to novel targets. Evaluations should measure validated outcomes rather than proxy metrics where possible, and should include assessment of false positives and over-defense costs.

When SAGE does not help. Adaptation provides value only when the environment exhibits exploitable structure. If a single architecture performs well across all conditions (no regime variation), SAGE adds overhead without benefit. If regime changes are unpredictable or too fast to detect, learning cannot track them. If the safety constraint is so tight that only one template is feasible, there is nothing to adapt. Our sensitivity analysis shows this explicitly: at $\kappa = 0.04$, SAGE matches the Fixed policy. These conditions describe when static architectures suffice, and practitioners should evaluate whether their deployment context exhibits the regime variation and binding constraints that make adaptation valuable.

10.3 Responsible Deployment

Sage is designed exclusively for authorized defensive testing. The system should only be deployed against targets where the operator has explicit permission to conduct security testing, the scope and rules of engagement are clearly defined, tools and permissions are restricted to the minimum necessary, targets are appropriately isolated to prevent unintended impact, all actions are logged and auditable, and legal and compliance requirements are satisfied.

The techniques in this paper could in principle be misused for offensive purposes, but we have intentionally avoided providing exploit instructions, payload generation capabilities, or other content that would lower the barrier to misuse. The contribution of this work is a framework for architecture adaptation, not offensive capability development.

11 RELATED WORK

This section situates Sage within the broader landscape of related work, highlighting connections and differences with prior approaches. We organize the discussion by research area and conclude with a summary comparison in Table 5.

Tool-using and agentic language models. ReAct [33] establishes the paradigm of interleaved reasoning and action, demonstrating that explicit reasoning traces improve task completion on knowledge-intensive benchmarks. Toolformer [25] shows that models can learn to invoke tools through self-supervised training on generated examples. AutoGen [30] provides infrastructure for multi-agent conversation that enables complex coordination patterns. These frameworks provide the components that Sage orchestrates, but do not address the question of how to adapt orchestration to target-specific conditions. In contrast, Sage treats the graph topology and component selection as dynamic control variables.

Security-focused LLM systems. PentestGPT [6] decomposes pentesting into interacting modules and evaluates on test machines, demonstrating the viability of multi-component architectures for security tasks. Work on LLM-based exploitation [8] studies the offensive capabilities of language model agents. Sage complements these systems by providing a framework for adapting their architecture during deployment rather than using a fixed configuration.

Security evaluation suites. CyberSecEval [4, 19] measures prompt injection resistance, code interpreter abuse, and other risk categories. CyberSOCEval [22] targets security operations center workflows including malware analysis and threat intelligence. These benchmarks enable rigorous evaluation of Sage against standardized tasks and metrics, and we specify an evaluation protocol using them in Section 8.

Prompt injection defenses. Empirical studies [9, 16] demonstrate the prevalence and severity of prompt injection in deployed systems. The OWASP Top 10 for LLM Applications [23] codifies these risks for practitioners. Guard models [15, 18] provide detection capabilities that can be integrated into agent graphs. Sage's contribution is to make guard placement and architectural constraints first-class optimization targets, complementing but not replacing detection-based defenses.

Safe reinforcement learning. Constrained Policy Optimization [1] introduces trust-region updates with approximate constraint satisfaction guarantees. Primal-dual methods [7] achieve stronger convergence properties. Surveys [13, 34] summarize the landscape of techniques. Sage adapts these methods to the specific structure of architecture adaptation with graph-valued actions and security-specific constraints, adding hard constraints via projection that are absent in standard CMDP formulations.

Contextual bandits and off-policy evaluation. LinUCB [5, 14] provides regret guarantees for linear reward models. Doubly robust estimation [10] enables off-policy evaluation from logged data. Sage applies these techniques to architecture selection, extending them with Lagrangian constraint handling.

Table 5: Comparison of Sage with related approaches along key dimensions.

Approach	Online adapt.	Safety constr.	Graph actions	Partial obs.	Security focus
ReAct/Toolformer	×	×	×	✓	×
PentestGPT	×	×	×	✓	✓
Neural Arch. Search	×	×	✓	×	×
Standard CMDP	✓	✓	×	×	×
Sage	✓	✓	✓	✓	✓

Neural architecture search. Zoph and Le [35] show that reinforcement learning can discover competitive neural network architectures for image classification. While the action space (layer specifications) and setting (offline optimization for a fixed task) differ substantially from Sage (online adaptation in a nonstationary environment), the conceptual precedent establishes that optimization over discrete graph structures is tractable.

Multi-agent system scaling. Recent work by Kim *et al.* [12] derives empirical scaling principles for multi-agent systems, evaluating five canonical architectures (single-agent, centralized, decentralized, independent, hybrid) across diverse benchmarks. They find that coordination benefits are task-contingent: centralized coordination improves performance by 80.9% on parallelizable tasks but degrades performance by 39–70% on sequential reasoning tasks. Their predictive model achieves $R^2 = 0.513$ for selecting the optimal architecture given task properties. This work is complementary to Sage: Kim *et al.* select the best *static* architecture at design time based on task category, while Sage *adapts* architecture dynamically during an engagement as conditions change. Their finding that no single architecture dominates across all tasks motivates online adaptation—if the engagement transitions between task types (e.g., from reconnaissance to exploitation), the optimal architecture may shift, requiring the kind of runtime adaptation Sage provides.

12 CONCLUSION

We have presented Sage, a constrained POMDP framework for online adaptation of agentic security architectures. The key insight is that architecture decisions—graph topology, component routing, and model-tool configuration—can be treated as first-class actions in a sequential decision problem, optimized under explicit budget and safety constraints.

The framework separates concerns between hard structural constraints that hold by construction regardless of learning dynamics and soft expected-risk constraints that are enforced through Lagrangian optimization. This two-layer design provides defense in depth: even if prompt injection or other attacks compromise model behavior, architectural constraints limit the blast radius.

We have developed learning algorithms spanning the spectrum from constrained contextual bandits for stage-wise decisions to full constrained reinforcement learning for continuous adaptation, with offline training and off-policy evaluation hooks that enable learning from logged engagements. A reproducible synthetic case study demonstrates that safe adaptive policies can substantially outperform static designs under nonstationarity while maintaining safety guarantees.

The limitations of this work point toward concrete future directions: evaluation on real security benchmarks (we specify a protocol but do not execute it), robustification against distribution shift through domain randomization or meta-learning, integration with emerging LLM guardrails, extension to multi-agent settings, and theoretical analysis of regret bounds in our specific setting.

The key take-away is simple: *architecture adaptation is a learnable control problem.* When agentic security systems face nonstationary targets and binding safety constraints, treating graph topology and component selection as dynamic control variables—optimized under the same principled frameworks used for other sequential decision problems—can yield substantial gains over fixed designs. Sage provides both the formalization and the algorithms to make this concrete. As agentic security systems become more capable and more deployed, we hope this work spurs further research on adaptive, constrained, and safe automation for vulnerability discovery.

REFERENCES

[1] Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. 2017. Constrained Policy Optimization. In *International Conference on Machine Learning (ICML)*. 22–31.

[2] Eitan Altman. 1999. Constrained Markov Decision Processes. In *CRC Press*.

[3] Sanae Amani, Mahnoosh Alizadeh, and Christos Thrampoulidis. 2019. Linear Stochastic Bandits Under Safety Constraints. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[4] Manish Bhatt, Sahana Chennabasappa, Yue Li, Cyrus Nikolaidis, Daniel Song, Shengye Wan, Faizan Ahmad, Cornelius Aber, Raymond Chen, Dawn Jiang, et al. 2024. CyberSecEval 2: A Wide-Ranging Cybersecurity Evaluation Suite for Large Language Models. *arXiv preprint arXiv:2404.13161* (2024).

[5] Wei Chu, Lihong Li, Lev Reyzin, and Robert E Schapire. 2011. Contextual Bandits with Linear Payoff Functions. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*. 208–214.

[6] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. 2024. PentestGPT: An LLM-empowered Automatic Penetration Testing Tool. In *USENIX Security Symposium*.

[7] Dongsheng Ding, Kaiqing Zhang, Tamer Başar, and Mihailo R Jovanovic. 2024. Last-Iterate Convergent Policy Gradient Primal-Dual Methods for Constrained MDPs. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[8] Richard Fang, Rohan Bindu, Akul Gupta, Qiusi Zhan, and Daniel Kang. 2024. LLM Agents can Autonomously Hack Websites. In *arXiv preprint arXiv:2402.06664*.

[9] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *ACM ASIA Conference on Computer and Communications Security*.

[10] Nan Jiang and Lihong Li. 2016. Doubly Robust Off-policy Value Evaluation for Reinforcement Learning. In *International Conference on Machine Learning (ICML)*. 652–661.

[11] Leslie Pack Kaelbling, Michael L Littman, and Anthony R Cassandra. 1998. Planning and Acting in Partially Observable Stochastic Domains. *Artificial Intelligence* 101, 1-2 (1998), 99–134.

[12] Yubin Kim, Ken Gu, Chanwoo Park, Chunjong Park, Samuel Schmidgall, A. Ali Heydari, Yao Yan, Zhihan Zhang, Yuchen Zhuang, Mark Malhotra, Paul Pu Liang, Hae Won Park, Yuzhe Yang, Xuhai Xu, Yilun Du, Shwetak Patel, Tim Althoff, Daniel McDuff, and Xin Liu. 2025. Towards a Science of Scaling Agent Systems. *arXiv preprint arXiv:2512.08296* (2025).

[13] Aditya Kushwaha et al. 2024. A Survey of Safe Reinforcement Learning and Constrained MDPs. *arXiv preprint arXiv:2403.xxxxx* (2024).

[14] Lihong Li, Wei Chu, John Langford, and Robert E Schapire. 2010. A Contextual-Bandit Approach to Personalized News Article Recommendation. In *Proceedings of the 19th International Conference on World Wide Web (WWW)*. 661–670.

[15] Ziqing Li, Christoph Treude, and Mansooreh Zahedi. 2024. PIGuard: A Prompt Injection Guardrail for Large Language Models. In *arXiv preprint arXiv:2405.xxxxx*.

[16] Yi Liu, Gelei Deng, Zhengzi Xu, Yuekang Li, Yaowen Zheng, Ying Zhang, Lida Zhao, Tianwei Zhang, Kailong Wang, and Yang Liu. 2024. Prompt Injection attack against LLM-integrated Applications. In *USENIX Security Symposium*.

[17] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. 2017. Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[18] Hao Meng, Yulin Xu, Lida Zhao, et al. 2024. InjecGuard: Benchmarking and Mitigating Over-defense in Prompt Injection Guardrail Models. In *arXiv preprint arXiv:2410.xxxxx*.

[19] Meta AI. 2024. CyberSecEval 3: Advancing the Evaluation of Cybersecurity Risks and Capabilities in Large Language Models. *arXiv preprint arXiv:2408.01605* (2024).

[20] Andrew Y Ng, Daishi Harada, and Stuart Russell. 1999. Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping. In *International Conference on Machine Learning (ICML)*. 278–287.

[21] OpenAI. 2024. Introducing SWE-bench Verified. https://openai.com/index/introducing-swe-bench-verified/. (2024).

[22] Giuseppe Orlando, Xiaoyuan Chen, et al. 2024. CyberSOCEval: A Large-Scale Benchmark for Evaluating LLMs in Security Operations Center Workflows. *arXiv preprint arXiv:2407.xxxxx* (2024).

[23] OWASP Foundation. 2023. OWASP Top 10 for Large Language Model Applications. https://owasp.org/www-project-top-10-for-large-language-model-applications/. (2023).

[24] Yujia Qin, Shihao Liang, Yining Ye, et al. 2023. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. *arXiv preprint arXiv:2307.16789* (2023).

[25] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[26] David Silver and Joel Veness. 2010. Monte-Carlo Planning in Large POMDPs. *Advances in Neural Information Processing Systems (NeurIPS)* 23 (2010).

[27] Theodore R Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L Griffiths. 2024. Cognitive Architectures for Language Agents. *arXiv preprint arXiv:2309.02427* (2024).

[28] Qiaoyu Tang, Ziliang Deng, Hongyu Lin, Xianpei Han, Qiao Liang, and Le Sun. 2023. ToolAlpaca: Generalized Tool Learning for Language Models with 3000 Simulated Cases. *arXiv preprint arXiv:2306.05301* (2023).

[29] Lei Wang, Chen Ma, Xueyang Feng, et al. 2024. A Survey on Large Language Model based Autonomous Agents. *arXiv preprint arXiv:2308.11432* (2024).

[30] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework. *arXiv preprint arXiv:2308.08155* (2023).

[31] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, et al. 2023. The Rise and Potential of Large Language Model Based Agents: A Survey. *arXiv preprint arXiv:2309.07864* (2023).

[32] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Liber, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *arXiv preprint arXiv:2405.15793*.

[33] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.

[34] Weiye Zhao, Tairan He, Rui Chen, Tianhao Wei, and Changliu Liu. 2023. State-wise Safe Reinforcement Learning: A Survey. *arXiv preprint arXiv:2302.03122* (2023).

[35] Barret Zoph and Quoc V Le. 2017. Neural Architecture Search with Reinforcement Learning. In *International Conference on Learning Representations (ICLR)*.

A NOTATION SUMMARY

Table 6 summarizes the main notation used throughout the paper.

Table 6: Summary of notation.

Symbol	Description
$G_t = (V_t, E_t, \Theta_t)$	Architecture graph at time t
V_t	Node set (agents and tools)
E_t	Edge set (typed channels)
Θ_t	Parameters (model variants, timeouts, etc.)
τ_t	Execution trace up to time t
$\phi(\tau_t)$	Trace feature extractor
m_t	Budget and risk meters
$o_t = \psi(\tau_t, m_t)$	Observation available to controller
b_t	Belief state
a_t	Proposed edit action
a'_t	Projected safe edit
$\mathcal{G}_{\text{safe}}$	Admissible architecture set
$\Pi_{\mathcal{S}}$	Projection operator onto $\mathcal{G}_{\text{safe}}$
U_t	Security utility signal
C_t	Cost signal
R_t^{risk}	Risk signal
κ	Risk budget
B	Cost budget
λ	Lagrange multiplier for risk constraint
γ	Discount factor
π	Adaptation policy

B SAGE ADAPTER LOOP PSEUDOCODE

Algorithm 1 presents the core adaptation loop in pseudocode form. The algorithm interleaves engagement execution with architecture updates, maintaining belief state over the hidden regime and adjusting the Lagrange multiplier to enforce risk constraints.

Algorithm 1 SAGE Adapter Loop

Input: Initial architecture G_0, belief b_0, Lagrange multiplier λ_0
Input: Risk constraint κ, learning rate η
for $t = 0, 1, \ldots, T$ **do**
  Execute engagement step with G_t
  Collect trace increment $\Delta\tau_t$ and meter update Δm_t
  Compute observation $o_t = \psi(\Delta\tau_t, \Delta m_t)$
  Update belief $b_t = \text{Filter}(b_{t-1}, a_{t-1}, o_t)$
  Propose edit $a_t \sim \pi(\cdot \mid b_t)$
  Project to safe edit $a'_t = \Pi_{\mathcal{S}}(a_t)$
  Apply edit: $G_{t+1} = \text{ApplyEdit}(G_t, a'_t)$
  Compute signals $(U_t, C_t, R_t^{\text{risk}})$
  Update policy π using $(b_t, a'_t, U_t - \lambda_t R_t^{\text{risk}})$
  Update multiplier $\lambda_{t+1} = \max(0, \lambda_t + \eta(R_t^{\text{risk}} - \kappa))$
end for

The algorithm maintains three key state variables: the current architecture graph G_t, the belief state b_t that summarizes uncertainty about the hidden regime, and the Lagrange multiplier λ_t that enforces the risk constraint. At each step, the system executes one step of the engagement using the current architecture, observes the resulting trace and meter updates, updates its belief about the regime, proposes an edit action sampled from the current policy, projects that action to ensure it satisfies hard constraints, applies the projected edit to obtain the next architecture, and then updates both the policy and the Lagrange multiplier based on the observed signals.

Practical implementation. The MDP formalization guides algorithm design but is not "solved" at runtime. In practice, the policy π is a lightweight learned model (e.g., linear weights for Safe-LinUCB-Lag, or a small neural network for the RL variant) that maps the current observation to an action in milliseconds. The computationally expensive work—learning the policy weights—happens offline on logged data or in simulation. At runtime, SAGE operates as a simple control loop: every k engagement steps (or k minutes), extract features, query the policy, project the proposed edit, apply it, and update λ. The overhead is negligible compared to the cost of LLM inference in the agent graph itself. This design enables deployment without requiring specialized MDP solvers or online planning.

The policy update on line 11 can be instantiated with various algorithms depending on the setting. For the constrained contextual bandit setting, the update maintains linear models of reward and risk and adjusts confidence bounds based on the new observation. For the full RL setting, the update performs a constrained policy gradient step using the advantage function estimated from the critic networks.

C FEATURE EXTRACTION DETAILS

This appendix provides additional detail on the trace features used by the adaptation controller.

Coverage features are computed from instrumentation on the target system and testing tools. Line coverage counts the number of distinct source lines executed. Branch coverage counts distinct branch outcomes taken. Path coverage tracks distinct execution paths through the control flow graph. Endpoint coverage tracks API endpoints or protocol states reached. Contract coverage tracks preconditions, postconditions, and invariants exercised.

Finding features are computed from the output of analysis and validation components. Unique finding count is the number of distinct potential vulnerabilities reported. Validated PoC count is the number of findings for which an exploit proof of concept has been confirmed. Duplicate rate is the fraction of reported findings that are duplicates of earlier findings. False positive rate is the fraction of findings that fail validation.

Efficiency features capture resource utilization. Cost per validated issue is the total cost in tokens or time divided by the validated finding count. Mean time to validation is the average duration between initial finding report and validation completion.

Safety features aggregate alerts from guards and monitors. Prompt injection alert count is the number of times guards flag potential injection content. Exfiltration block count is the number of times outbound data transfers are blocked by policy. Policy violation count is the number of actions flagged as violating engagement rules.

Stability features detect when the adaptation algorithm is churning without progress. Oscillation count tracks how many times an edit is applied and then reverted within a window. Revert frequency is the fraction of edits that are subsequently reversed.

D SYNTHETIC ENVIRONMENT DETAILS

This appendix provides the full specification of the synthetic environment used in Section 9.

The environment has three regimes indexed by $z \in \{1, 2, 3\}$. The regime transition matrix is:

$$P_{\text{regime}} = \begin{pmatrix} 0.95 & 0.025 & 0.025 \\ 0.025 & 0.95 & 0.025 \\ 0.025 & 0.025 & 0.95 \end{pmatrix}, \tag{10}$$

so regimes are sticky with 95% probability of remaining in the current regime.

The six architecture templates have the following expected utility, cost, and risk per step, varying by regime. In regime 0 (validation bottleneck), templates 0 through 5 have utilities $(0.30, 0.48, 0.35, 0.32, 0.55, 0.45)$, costs $(0.62, 0.66, 0.55, 0.60, 0.68, 0.64)$, and risks $(0.04, 0.10, 0.12, 0.14, 0.06, 0.09)$. In regime 1 (false positive bottleneck), utilities are $(0.28, 0.40, 0.52, 0.30, 0.42, 0.48)$, costs are $(0.60, 0.62, 0.58, 0.58, 0.65, 0.62)$, and risks are $(0.04, 0.08, 0.05, 0.10, 0.14, 0.07)$. In regime 2 (high injection risk), utilities are $(0.32, 0.38, 0.35, 0.50, 0.36, 0.45)$, costs are $(0.58, 0.60, 0.55, 0.56, 0.62, 0.60)$, and risks are $(0.04, 0.18, 0.16, 0.05, 0.20, 0.1$ Note that template 0 is the only universally safe option with consistent low risk across all regimes, but it has poor utility. The best-performing templates in each regime have low risk only in that regime.

Observations are noisy indicators of the regime. The controller observes a 3-dimensional feature vector where each component is the regime's index plus Gaussian noise with standard deviation 0.5. This provides partial information about the regime that can be exploited by learning algorithms.

The risk constraint is $\kappa = 0.08$ average risk per step. The best static safe template, determined by computing expected risk under the stationary regime distribution, is template 0 (the only universally safe option) which has expected risk 0.04 and expected reward 0.020 when averaged over regimes with the transition-implied stationary distribution.